· CONCEPTION MICROSERVICES

SkyTrack

Plateforme de surveillance et d'analyse météorologique

6 microservices | Angular 21 | Spring Boot · .NET | PostgreSQL · MongoDB · InfluxDB

Équipe : Med Dhia Allagui · Yasmine Fehri · Ahmed Bouallegue · Abdallah Grissa · Yassine wesslati · Salim Khaldi ·

Le problème : une météo dispersée

Sources séparées

Prévisions, mesures des stations et alertes viennent d'outils différents.

Alertes génériques

Pas de notification ciblée par ville ni par utilisateur.

Pas d'historique

Les mesures passées ne sont pas agrégées pour être analysées.

SkyTrack réunit prévisions, mesures IoT, alertes et analyses dans une seule plateforme.

Acteurs et fonctionnalités

Utilisateur

- Consulter les prévisions d’une ville
- Recevoir les alertes
- Choisir °C ou °F
- Voir l’historique climatique

Administrateur

- Gérer utilisateurs et rôles
- Gérer villes et régions
- Publier les prévisions
- Déclencher les alertes

Station IoT

- Envoyer des mesures en continu
- Température, humidité, pluie
- Acteur système, sans humain

Découpage : une capacité métier par service

1 · .NET · :8080

User Service

Comptes, rôles et authentification

2 · Spring Boot · :8081

Location Service

Référentiel des villes et régions

3 · Spring Boot · :8082

Forecast Service

Prévisions météo par ville

4 · Spring Boot · :8083

Alert & Notification

Alertes climatiques et notifications

5 · Spring Boot · :8084

IoT Sensor

Mesures en temps réel des stations

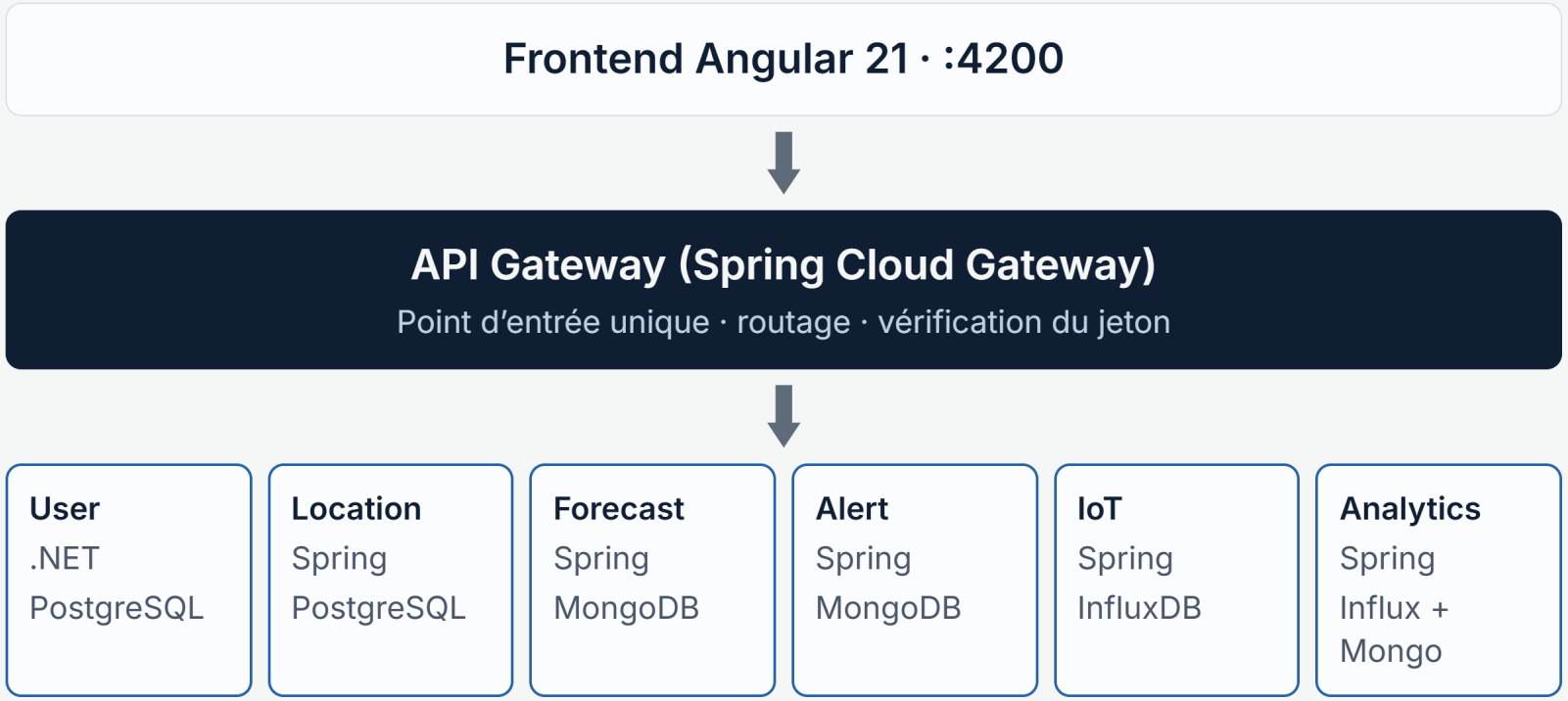
6 · Spring Boot · :8085

Analytics & Preferences

Historiques agrégés et préférences

Découpage par **capacité métier**, pas par table ni par écran : aucune responsabilité partagée.

Architecture technique



Keycloak

Authentification, jetons JWT

Eureka

Découverte des services par leur nom

Config Server

Configuration centralisée

Docker : chaque microservice, sa base et l'infrastructure tournent dans leur propre conteneur, lancés ensemble par docker-compose.

Données : une base par microservice

Microservice	Base	Pourquoi ce choix
User	PostgreSQL	Données structurées, relation User → Role
Location	PostgreSQL	Référentiel stable, relation City → Region
Forecast	MongoDB	Prévision et détail embarqué, souvent corrigés
Alert & Notification	MongoDB	Statut lu / non lu mis à jour
IoT Sensor	InfluxDB	Mesures horodatées écrites en continu
Analytics & Preferences	InfluxDB + MongoDB	Moyennes horodatées + préférences modifiables

Aucune base partagée : un service ne lit jamais la base d'un autre, il passe par son API.

Communication entre microservices

Appelant	Appelé	ID	Objectif
Forecast	Location	cityId	Nom de la ville d'une prévision
Alert & Notification	Location	cityId	Ville touchée par une alerte
Alert & Notification	User	userId	Destinataire d'une notification
Analytics & Preferences	Location	cityId	Ville d'un historique climatique
Analytics & Preferences	User	userId	Propriétaire des préférences

Synchrone, via OpenFeign

Chaque appel a besoin de la réponse tout de suite. Seuls des IDs sont échangés.

Si un service tombe

Timeout + circuit breaker (Resilience4j) : réponse en mode dégradé au lieu d'une erreur en cascade.

MICROSERVICE 1 · [AHMED BOUALLEGUE]

User Service

Rôle

Comptes et rôles

Technologie

ASP.NET Core (.NET)

Base de données

PostgreSQL · :5432

Port de l'API

8080

Entités

User

id · username · email · password · roleId

N..1

possède

Role

id · name

Liens avec les autres services

Appelé par Alert & Notification et Analytics (userId). N'appelle aucun service. User.Id est un long en C#, comme le Long côté Java.

MICROSERVICE 2 · [SELIM KHALDI]

Location Service

Rôle

Référentiel des villes

Technologie

Spring Boot

Base de données

PostgreSQL · :5433

Port de l'API

8081

Entités

City

id · name · latitude · longitude · regionId

N..1

appartient à

Region

id · name · country

Liens avec les autres services

Appelé par Forecast, Alert & Notification et Analytics (cityId). Service de référence : il n'appelle aucun autre service.

MICROSERVICE 3 · MED DHIA ALLAGUI

Forecast Service

Rôle

Prévisions par ville

Technologie

Spring Boot

Base de données

MongoDB · :27017

Port de l'API

8082

Entités

Forecast

id · cityId · date · summary

1..1

contient (embarqué)

WeatherDetail

temperature · humidity · windSpeed · pressure

Liens avec les autres services

Appelle Location (cityId → nom de la ville) via OpenFeign. Si Location tombe : lecture en mode dégradé, création refusée (503).

MICROSERVICE 4 · [YASMINE FEHRI]

Alert & Notification Service

Rôle

Alertes et notifications

Technologie

Spring Boot

Base de données

MongoDB · :27018

Port de l'API

8083

Entités

Alert

id · cityId · type · severity · createdAt

1..*

déclenche

Notification

id · alertId · userId · message · read

Liens avec les autres services

Appelle Location (cityId) et User (userId) via OpenFeign. Le champ read d'une notification passe de non lu à lu.

MICROSERVICE 5 · [YASSINE WESSLATI]

IoT Sensor Service

Rôle

Mesures des stations

Technologie

Spring Boot

Base de données

InfluxDB · :8086

Port de l'API

8084

Entités

Station

stationId · name · cityId

1..*

envoie en continu

SensorMeasurement

timestamp · stationId · temperature · humidity · rain

Liens avec les autres services

Service autonome : aucun appel vers les autres. Reçoit les mesures des stations, stockées en points InfluxDB (tag stationId).

MICROSERVICE 6 · [ABDALLAH GRISSA]

Analytics & Preferences

Rôle

Historiques et préférences

Technologie

Spring Boot

Bases de données

InfluxDB :8087 + MongoDB :27019

Port de l'API

8085

Entités

HistoricalLog

cityId · period · avgTemperature · avgHumidity

InfluxDB

2 bases

sans relation

UserPreference

id · userId · unit · alertsEnabled

MongoDB

Liens avec les autres services

Appelle Location (cityId) pour l'historique d'une ville et User (userId) pour le propriétaire des préférences, via OpenFeign.

Dépôt Git et organisation

```
skytrack/
├── Backend/
│   ├── forecast-service/
│   └── … un projet par service
├── Frontend/
├── Documentation/
│   ├── Presentation/
│   └── Diagrams/
└── README.md
```

- Une **branche par membre**
- **Pull request** vers main, jamais de push direct
- Commits réguliers de **chaque membre**
- Suivi des tâches : **GitHub Projects**

github.com/Ahmedbouallegue/skytrack_microservice_project-

État et prochaines étapes

Fait

- Sujet, acteurs et fonctionnalités
- Découpage et diagramme de classes
- Dépôt Git structuré
- Projets des microservices créés

Après validation

- Logique métier de chaque service
- Gateway, Eureka, Config, Keycloak
- docker-compose de l'ensemble
- Frontend Angular

Merci. Des questions ?